Analysis of Algorithm

1. **Time Complexity**
2. **Space Complexity**
3. **Asymptotic notations**
   - **Big O**
   - **Big Theta**
   - **Big Omega**

Analysis of Algorithm

Space complexity:

- An algorithm's **space complexity** is the amount of space required to solve a problem and produce an output. Similar to the time complexity, space complexity is also expressed in big O notation.
- **Why Space is required?**
  - To store program instructions
  - To store constant values
  - To store variable values
  - To track the function calls, jumping statements, etc

  Space complexity = Auxiliary space + Input size.
  - **Auxiliary space**: The extra space required by the algorithm, excluding the input size, is known as an auxiliary space.
- *Total amount of computer memory required by an algorithm to complete its execution is called as **space complexity** of that algorithm.*
- When a program is under execution it uses the computer memory for THREE reasons
  1. **Instruction Space:** It is the amount of memory used to store compiled version of instructions.
  2. **Environmental Stack:** It is the amount of memory used to store information of partially executed functions at the time of function call.
  3. **Data Space:** It is the amount of memory used to store all the variables and constants.

Analysis of Algorithm

Asymptotic notations

- **Big O**
- **Big Theta**
- **Big Omega**

Complexity Analysis

- Complexity analysis is the systematic study of the cost of a computation, measured either in time units or in operations performed, or in the amount of storage space required.
- The goal is to have a meaningful measure that permits comparison of algorithms and/or implementations independent of operating platform.
- Complexity analysis involves two distinct phases:
  - **algorithm analysis**: analysis of the algorithm or data structure to produce a function T(n) measuring the complexity
  - **order of magnitude (asymptotic) analysis**: analysis of the function T(n) to determine the general complexity category to which it belongs.

- Algorithm analysis requires a set of rules to determine how operations are to be counted.
- There is no generally accepted set of rules for algorithm analysis.
- In some cases, an exact count of operations is desired; in other cases, a general approximation is sufficient.

Complexity Analysis

Running time of an algorithm

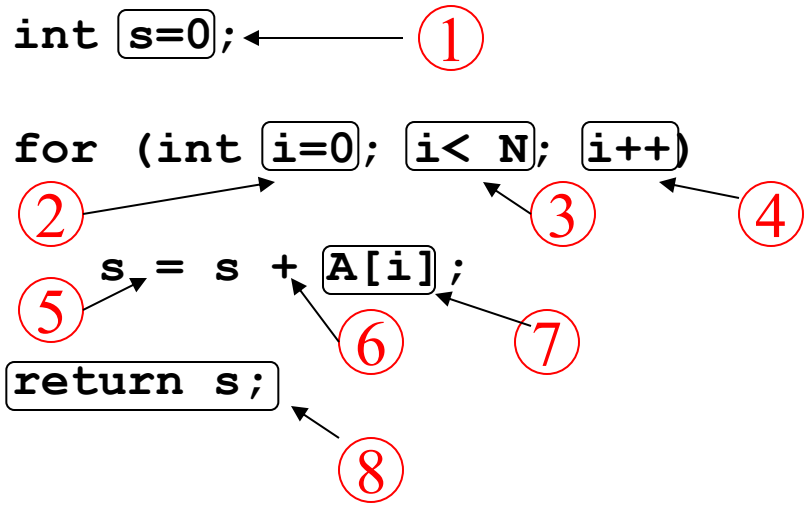
- Depends upon
    - Input Size
    - Nature of Input
- Generally time grows with size of input, so running time of an algorithm is usually measured as function of input size.
- Running time is measured in terms of number of steps/primitive operations performed
- Independent from machine, OS
- Running time is measured by number of steps/primitive operations performed
- Steps means elementary operation like
  ,+, *,<, =, A[i] etc
- We will measure number of steps taken in term of size of input

Complexity Analysis

Running time of an algorithm (Example)

```
// Input: int A[N], array of N integers
// Output: Sum of all numbers in array A

int Sum(int A[], int N){
    int s=0;
    for (int i=0; i< N; i++)
        s = s + A[i];
    return s;
}
```



1,2,8: Once
3,4,5,6,7: Once per each iteration of for loop, N iteration
Total: 5N + 3
The *complexity function* of the algorithm is : $f(N) = 5N + 3$

Complexity Analysis

COMPARING FUNCTIONS: ASYMPTOTIC NOTATION

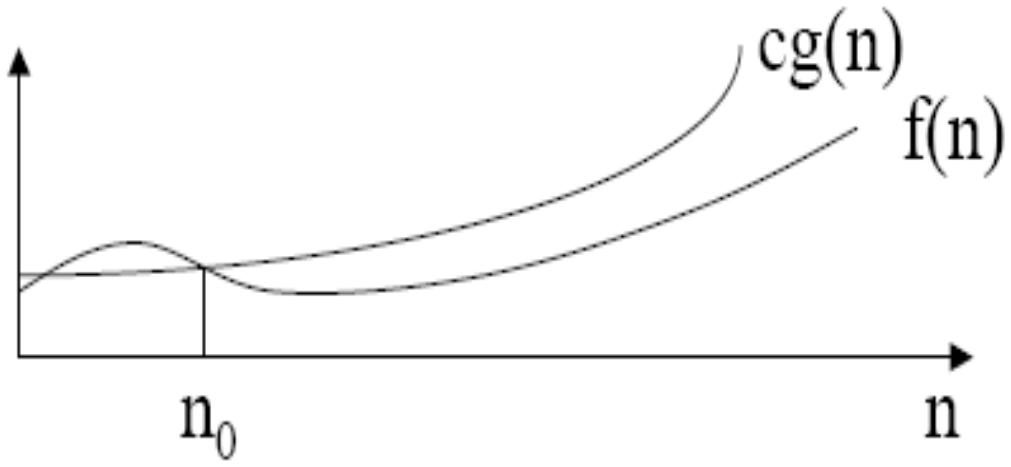
- Big Oh Notation: Upper bound
- Omega Notation: Lower bound
- Theta Notation: Tighter bound

Complexity Analysis

Big Oh Notation

- Big oh(O) notation is used to represent upperbound of algorithm runtime.
- Let f(n) and g(n) are two non-negative functions. The function f(n) = O(g(n)) if and only if there exists positive constants c and n_0 such that f(n)≤c*g(n) for all n , n ≥ n_0.
- In Other words: If f(n) and g(n) are two complexity functions, we say f(n) = O(g(n)) *(read "f(n) is order g(n)", or "f(n) is big-O of g(n)")* if there are constants c and n_0 such that for n ≥ n_0,

f(n) ≤ c * g(n) for all sufficiently large n.



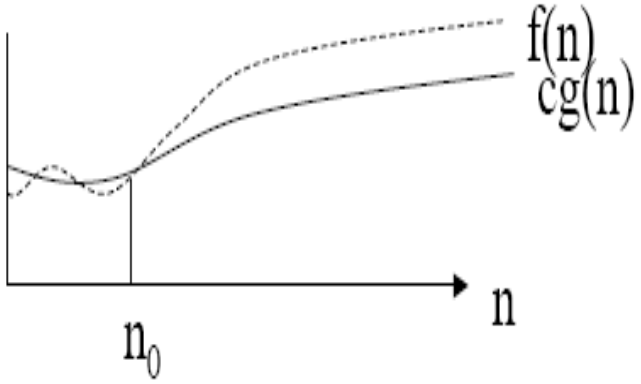
- Function cg(n) always dominates f(n) to the right of n_0

Complexity Analysis

Big Omega Notation

- If we wanted to say "running time is at least…" we use Ω
- Big Omega notation, Ω, is used to express the lower bounds on a function.
- If f(n) and g(n) are two complexity functions then we can say:

f(n) is Ω(g(n)) if there exist positive numbers c and n_0 such that 0<=f(n)>=cΩ (n) for all n ≥ n_0



- In this instance, function cg(n) is dominated by function f(n) to the right of n_0

- Example : 3n + 2 = Ω(n)

f(n)=3n+2 then prove that f(n) = Ω(g(n))

Let f(n) =3n+2, c=3, g(n) =n

if n=1, 3n+2 ≥ 3n

3(1)+2 ≥ 3(1)

5 ≥ 3 (T)

3n+2 ≥ 4n for all n ≥ 1

This is in the form of f(n) ≥ c*g(n) for all n ≥ n_0,

where c=3, n_0 =1

Therefore, f(n) = Ω(n).

Complexity Analysis

Big Theta Notation

- Theta(ɵ) notation is used to represent the running time between upper bound and lower bound.
- If we wish to express tight bounds we use the theta notation, Θ
- Let f(n) and g(n) be two non-negative functions.
- The function f(n) = θ(g(n)) if and only if there exists positive constants c1 , c2 and n_0 such that c_1*g(n) ≤ f(n)≤c_2* g(n) for all n, n≥n_0 .

f(n)=3n+2 then Prove that f(n) = θ(g(n))

Lower bound = 3n+2 ≥ 3n for all n ≥ 1

c_1=3,g(n)=n,n_0=1

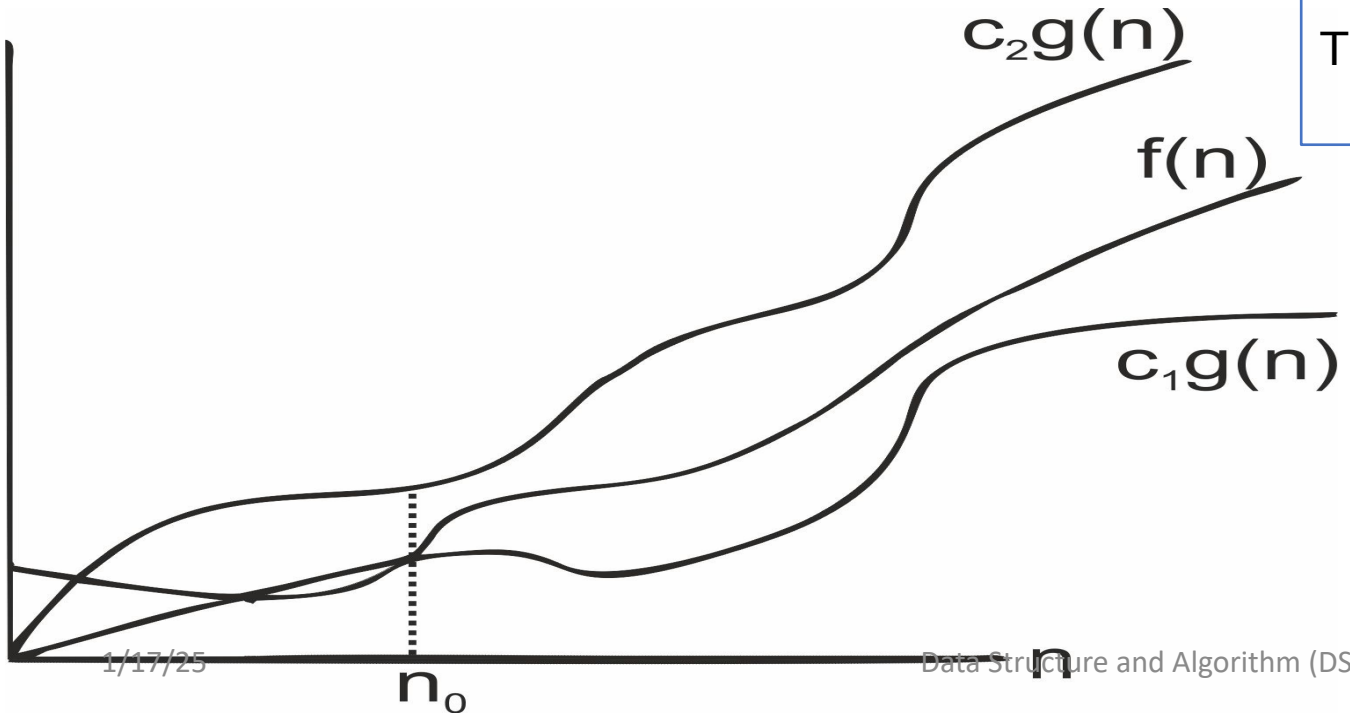
Upper Bound = 3n+2 ≤ 4n for all n ≥ 2

c_2=4, g(n)=n , n_0=2

3(n) ≤ 3n+2 ≤ 4(n) for all n, n ≥ 2

This is in the form of c_1*g(n) ≤ f(n) ≤ c_2* g(n) for all n≥n_0 Where c_1=3, c_2=4, g(n)=n, n_0=2

Therefore f(n)= θ(n)



Complexity Analysis

What does this all mean?

- If $f(n) = \Theta(g(n))$ we say that f(n) and g(n) grow at the same rate, asymptotically
- If $f(n) = O(g(n))$ and $f(n) \neq \Omega(g(n))$, then we say that f(n) is asymptotically slower growing than g(n).
- If $f(n) = \Omega(g(n))$ and $f(n) \neq O(g(n))$, then we say that f(n) is asymptotically faster growing than g(n).

Which Notation do we use?

- To express the efficiency of our algorithms which of the three notations should we use?
  - As computer scientist we generally like to express our algorithms as big O since we would like to know the upper bounds of our algorithms.
  - If we know the worse case then we can aim to improve it and/or avoid it.

Complexity Analysis

Performance Classification

f(*n*)	Classification
1	***Constant***: run time is fixed, and does not depend upon n. Most instructions are executed once, or only a few times, regardless of the amount of information being processed
log n	***Logarithmic***: when *n* increases, so does run time, but much slower. Common in programs which solve large problems by transforming them into smaller problems. Exp : binary Search
n	***Linear***: run time varies directly with *n*. Typically, a small amount of processing is done on each element. Exp: Linear Search
n log n	When *n* doubles, run time slightly more than doubles. Common in programs which break a problem down into smaller sub-problems, solves them independently, then combines solutions. Exp: Merge
n^2	***Quadratic***: when *n* doubles, runtime increases fourfold. Practical only for small problems; typically the program processes all pairs of input (e.g. in a double nested loop). Exp: Insertion Search
n^3	***Cubic***: when n doubles, runtime increases eightfold. Exp: Matrix
2^n	***Exponential***: when n doubles, run time squares. This is often the result of a natural, "brute force" solution. Exp: Brute Force. Note: logn, n, nlogn, n^2>> less Input>>Polynomial n^3, 2^n>>high input>> non polynomial

Complexity Analysis

Standard Analysis Techniques

- Constant time statements
- Analyzing Loops
- Analyzing Nested Loops
- Analyzing Sequence of Statements
- Analyzing Conditional Statements

Complexity Analysis

Constant time statements

- Simplest case: O(1) time statements
- Assignment statements of simple data types
  int x = y;
- Arithmetic operations:
  x = 5 * y + 4 - z;
- Array referencing:
  A[j] = 5;
- Array assignment:
  ∀ j, A[j] = 5;
- Most conditional tests:
  if (x < 12) ...

Analyzing Loops

- Any loop has two parts:
  - How many iterations are performed?
  - How many steps per iteration?

```
int sum = 0,j;
  for (j=0; j < N; j++)
    sum = sum +j;
```

  - Loop executes N times (0..N-1)
  - O(1) steps per iteration
- Total time is n * O(1) = O(n*1) = **O(n)**

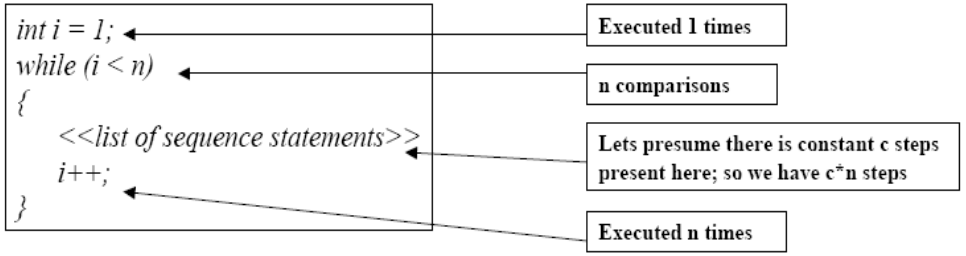
Analyzing Loops

- What about this **for** loop?

```
int sum =0, j;
for (j=0; j < 100; j++)
   sum = sum +j;
```

- Loop executes 100 times
- O(1) steps per iteration
- Total time is 100 * O(1) = O(100 * 1) = O(100) = **O(1)**
- Note: O(100) is a constant, which simplifies to **O(1)**

Example (have a look at this code segment):



- Efficiency is proportional to the number of iterations.
- Efficiency time function is :
  f(n) = 1 + (n-1) + c*(n-1) +(n-1)
  = (c+2)*(n-1) + 1
  = (c+2)n – (c+2) +1
- Asymptotically, efficiency is : **O(n)**

Complexity Analysis

Analyzing Nested Loops

- Treat just like a single loop and evaluate each level of nesting as needed:

```
int j,k;
for (j=0; j<N; j++)
   for (k=N; k>0; k--)
      sum += k+j;
```

- Start with outer loop:
  - How many iterations? N
  - How much time per iteration? Need to evaluate inner loop
- Inner loop uses O(N) time
- Total time is N * O(N) = O(N*N) = **$O(N^2)$**

- What if the number of iterations of one loop depends on the counter of the other?

```
int j,k;
for (j=0; j < N; j++)
  for (k=0; k < j; k++)
    sum += k+j;
```

- Analyze inner and outer loop together:
- Number of iterations of the inner loop is:
- 0 + 1 + 2 + ... + (N-1) = **$O(N^2)$**

Complexity Analysis

Conditional Statements

```
if (x > 0)
    if (x % 2 == 0)
        for (i = 0; i < n; i++)
        sum += i;   // O(n)
    else
        sum = 1;   // O(1)
else
    y = x - 1;     // O(1)
```

- Condition evaluations: O(1) for each condition.
- Branch complexities: O(n),O(1),O(1).
- Total complexity: **O(n)**.

```
if (arr[mid] == key)
    return mid;   // O(1)
else if (arr[mid] < key)
    low = mid + 1;   // O(1)
else
    high = mid - 1;   // O(1)
```

- Condition evaluations: O(1) for comparisons.
- Statement execution: All operations inside each branch are O(1).
- In a loop (e.g., binary search), the complexity depends on the number of iterations: **O(logn)**

Key Points for Analysis

1.Evaluate the **complexity of the condition**.
2.Assess the **complexity of each branch**.
3.The overall complexity is determined by:

- O(condition complexity)+O(max (branch complexities))

Complexity Analysis

Big-O Notation (Common Complexities)

- $T(n)=O(1)$ // constant time
- $T(n)=O(\log n)$ // logarithmic
- $T(n)=O(n)$ // linear
- $T(n)=O(n^2)$ //quadratic
- $T(n)=O(n^3)$ //cubic
- $T(n)=O(n^c)$, $c \geq 1$ // polynomial
- $T(n)=O(\log^c n)$, $c \geq 1$ // polylogarithmic
- $T(n)=O(n\log n)$

Few Examples

```
if(Condition)                    c1
{
        printf("Hi");            c2
}
else{                            c3
        printf("Hello");         c4
}
```

Breaking down the code
IF condition cost = C1 (Executes once)
IF statement cost = C2 (Executes if IF is true)
ELSE condition cost = C3 (Executes once)
ELSE statement cost = C4 (Executes if IF is false)
Now there are two possibilities:

1. If IF condition is true
   IF condition cost = C1
   IF statement cost = C2
2. Total cost = C1 + C2

3. If IF condition is false
   IF condition cost = C1
   ELSE condition cost = C3
   ELSE statement cost = C4
4. Total cost = C1 + C3 + C4

Since if condition can only be either true or false, the total cost will be:
Total Cost = Max(C1 + C2, C1 + C3 + C4)
By taking the max, we consider the costlier of the two possibilities (IF true or IF false).
Simplified, the total cost is:
Total Cost = Max(IF Cost, ELSE Cost)
Where:
IF Cost = C1 + C2
ELSE Cost = C1 + C3 + C4
So we take the maximum between the IF section cost or the ELSE section cost as the total cost.
It has constant time complexity O(1), as the costs C1, C2, C3, C4 are fixed, not dependent on any input size parameter.

Calculate the time complexity for this snippet

```
for(i=1;i<=M;i++){              C1
 for(kj=1;j<=N;j++){            C2
        printf("Hello");     C3
}
}
```

Analyzing step by step
Outer loop cost per iteration = C1
Inner loop cost per iteration = C2
Instruction inside inner loop cost per iteration = C3
Let's assume:
- Outer loop runs M iterations
- Inner loop runs N iterations per each outer iteration

Then, costs will be:
1. Outer loop
   - Runs M iterations
   - Cost = M * C1
2. Inner loop
   - Runs N iterations
   - Runs M times (once per outer loop iteration)
   - Cost per inner loop = N * C2
   - Total inner loops = M (outer iterations) : So total cost = M * (N * C2) = M * N * C2
3. Instructions inside inner loop
   - Have cost C3 per iteration
   - Run N times per inner loop
   - With M outer loops
   - So total times instructions execute = M * N
   - Hence total cost = M * N * C3
4. Total Cost = Outer Loop Cost + Inner Loop Cost + Instruction

Cost = M * C1 + M * N * C2 + M * N * C3
Total Cost = M(C1 + NC2 + NC3)

This shows a polynomial time complexity of **O(M*N)** for nested loops.

Calculate time complexity

```
for(i=1;i<=N;i++){                C
        If(condition){            C1
                Statements        C2
        }
        else{                     C3
                Statements        C4
        }
}
```

- Cost of for loop overhead = C (initialization, condition, update statements)
- If-else costs:
  - IF condition cost = C1
  - IF statement cost = C2
  - ELSE condition cost = C3
  - ELSE statement cost = C4

Let the for loop run N iterations

Then, cost per iteration = Max(C1 + C2, C1 + C3 + C4) (If-else cost)

And the for loop runs N iterations.

Therefore, total cost = Cost of N for loop iterations + N times (If-else cost per iteration)

Total Cost = C + N * Max(C1 + C2, C1 + C3 + C4)

Simplifying: Total Cost = C + N * Max(IF Cost, ELSE Cost)

Where:

IF Cost = C1 + C2

ELSE Cost = C1 + C3 + C4

N = number of iterations

So overall the total cost is linear in terms of N (number of iterations).

Hence, the overall time complexity with if-else inside loop is **O(N)**.

Question

```
m=1                          C1 cost
for(i=1;i<=n;i++){           C2 cost
        for(j=1;j<=n;j++){   C3 cost
                m=i+j;       C4 cost
        }
}
return m;                    C5 cost
```

Counting executions per statement:

1. m = 1 (once) : Cost = C1
2. Outer loop initialization (i = 1): Cost = C2
3. Outer loop test (i <= n): Runs n+1 times Cost = (n+1)*C2
4. Inner loop initialization (j = 1): Runs n times (once per outer iteration) Cost = n*C3
5. Inner loop test (j <= n):

   Runs n^2 times Cost = n^2*C3
6. m = i + j: Runs n^2 times Cost = n^2*C4
7. Return statement: Runs once Cost = C5

Total Cost = C1 + (n+1)C2 + nC3 + n^2C3 + n^2C4 + C5

= C1 + C2 + C3 + 2n^2C3 + n^2*C4 + C5

This is **O(n^2)** time complexity dominated by the nested loop iterations.

In the worst case, we consider the maximum number of iterations for each loop. For each iteration of the outer loop, the inner loop runs n times.

Total cost for worst case = C1 * n + C2 * n * C3 * n + C4 * n * n + C5

n the average case, we would typically consider the average number of operations. Since the code does not contain any conditional statements or variable input sizes, the average case is the same as the worst case.

Total cost for average case = C1 * n + C2 * n * C3 * n + C4 * n * n + C5

Therefore, both the worst case and average case time complexity for the given code are O(n^2)

More Examples

```
int j, k, l;
for (j = 0; j < N; j++)
   for (k = 0; k < j; k++)
      sum += k + j;

for (l = 0; l < N; l++)
   sum += l;
```

Step	Explanation	Number of Iterations	Complexity
Step 1: Outer and inner loops (`j` and `k`)	Outer loop runs `N` times, inner loop runs from `0` to `j - 1`. Total iterations for inner loop: $\frac{N(N-1)}{2}$.	$\frac{N(N-1)}{2}$	**O(N²)**
Step 2: Second loop (`l`)	A single loop that runs `N` times.	N	**O(N)**

Total Complexity

The complexities of **both parts** are added together:

$$O(N^2) + O(N) = O(N^2) \quad \text{(since } N^2 \text{ dominates over } N \text{ for large inputs)}.$$

Step	Explanation
Initialization	`i = 1` and `sum = 0`.
Loop Condition	The loop runs while `i < N`.
Loop Body	Each iteration adds `i` to `sum`. The key step is updating `i` to `i * 2` (multiplying `i` by 2 in each iteration).
Number of Iterations	`i` starts at 1 and grows exponentially: `1, 2, 4, 8, 16, ...` until it reaches or exceeds `N`.

```
sum = 0;
for (i = 1; i < N; i = i * 2)
    sum = sum + i;
```

How Many Iterations?

Since `i` doubles in every iteration, this forms a geometric progression. The values of `i` will be:

- 1, 2, 4, 8, ..., up to the largest power of 2 less than `N`.

Let's denote the number of iterations as **k**. The value of `i` after **k** iterations is:

$$i = 2^k$$

The loop terminates when **i ≥ N**, so:

$$2^k < N \quad \Rightarrow \quad k = \log_2 N$$

Time Complexity

- The number of iterations is **k = $\log_2$ N**, so the time complexity is:

$$O(\log N)$$

```
for (i = N; i >= 1; i = i / 2)
   statements
```

Number of Iterations:

To calculate the number of iterations until the loop terminates, let's consider the following:

- Initially, `i = N`.
- After the first iteration, `i = N / 2`.
- After the second iteration, `i = N / 4`.
- After the third iteration, `i = N / 8`, and so on.

The loop will continue until `i` becomes less than 1. We can express this mathematically as:

$$i = \frac{N}{2^k}$$

where k is the number of iterations. The loop stops when $i < 1$, so we need to find the value of k when:

$$\frac{N}{2^k} < 1$$

Multiplying both sides by 2^k:

$$N < 2^k$$

Taking the logarithm (base 2) of both sides:

$$\log_2(N) < k$$

So, the number of iterations k is:

$$k = \lceil \log_2(N) \rceil$$

Thus, the number of iterations is approximately $\lceil \log_2(N) \rceil$, where $\lceil x \rceil$ denotes the ceiling function, which rounds x to the next integer.

The time complexity is **O(logN)**

```
for (i = 0; i < n; i = i++)
    for (j = 1; j < n; j = j * 2)
        statements
```

- The outer loop runs $O(n)$ times.
- The inner loop runs $O(\log n)$ times for each iteration of the outer loop.

Therefore, the total time complexity of the nested loops is:

$$O(n) \times O(\log n) = O(n \log n)$$

Summary

Loop Type	Code Structure	Time Complexity	Explanation
Simple Increment (Linear)	`for (i = 0; i < n; i++)`	$O(n)$	Runs `n` times, increments by 1 each iteration.
Geometric Progression	`for (i = 1; i < n; i = i * 2)`	$O(\log n)$	Doubles the value of `i` in each iteration. Runs logarithmically.
Quadratic (Nested Loops)	`for (i = 0; i < n; i++)` `for (j = 0; j < n; j++)`	$O(n^2)$	Two nested loops, each running `n` times.
Logarithmic (Nested)	`for (i = 0; i < n; i++)` `for (j = 1; j < n; j = j * 2)`	$O(n \log n)$	Outer loop runs `n` times, inner loop runs logarithmically (`\log n` times).
Exponential Progression	`for (i = 1; i < n; i = i * i)`	$O(\log \log n)$	Each iteration squares `i`, leading to very few iterations.
Factorial Progression	`for (i = 1; i <= n; i++)` `for (j = 1; j <= i; j++)`	$O(n^2)$	Inner loop depends on the outer loop variable, leading to quadratic behavior.

Summary

Loop Type	Code Structure	Time Complexity	Explanation
Cubic Nested Loops	`for (i = 0; i < n; i++)` `for (j = 0; j < n; j++)` `for (k = 0; k < n; k++)`	$O(n^3)$	Three nested loops, each running `n` times.
Increments by Constant	`for (i = 0; i < n; i += c)` (where `c` is a constant)	$O(n)$	Same as simple increment but increments by a constant `c`.
Non-Linear Nested Loops	`for (i = 0; i < n; i++)` `for (j = i; j < n; j++)`	$O(n^2)$	Nested loop with variable bounds depending on `i`.
Binary Search (Divide and Conquer)	`for (i = 1; i < n; i = i * 2)`	$O(\log n)$	Similar to geometric progression, often used in algorithms like binary search.

Key Observations:

- **Linear** loops run a fixed number of times, directly proportional to n.
- **Logarithmic** loops halve or exponentially decrease the range, reducing iterations significantly.
- **Quadratic** loops occur when there are two levels of iteration over n.
- **Cubic** or higher-order nested loops arise from multiple levels of iteration, leading to exponential growth in complexity.

Thank you